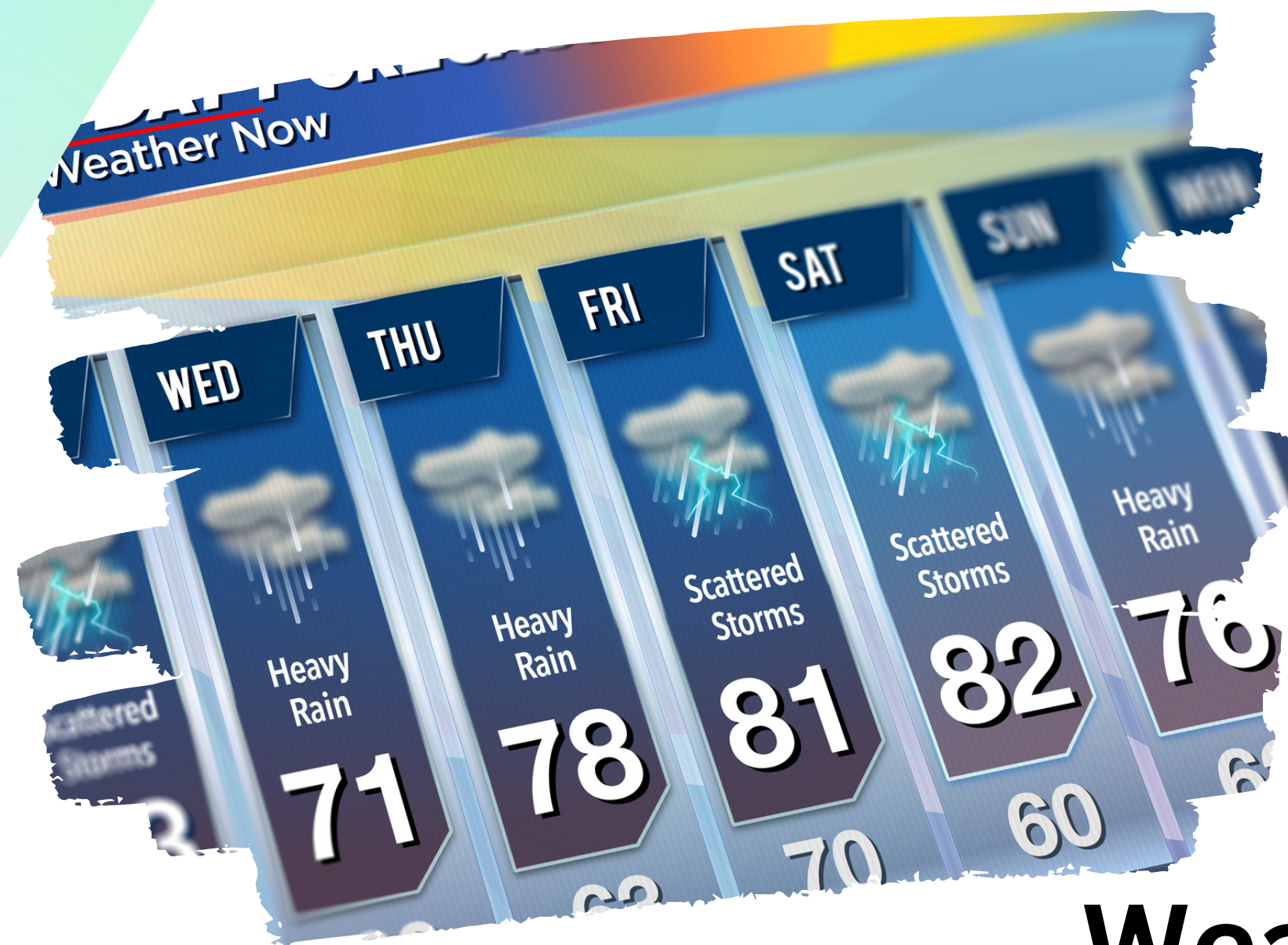




Weather Forecasting Using Deep Learning (RNN/LSTM)

S.C.L Fernando
SC/10477
Meteorology Department

Importance of Weather Forecasting

Significance in daily life

eg: planning outdoor events
travel arrangements



Agriculture and Farming

eg :Planting
Harvesting



Disaster preparedness and management

eg :Preparing for natural disasters like flooding, tornado etc.



Forecasting weather is a challenging task.

Types of weather data

- Weather Forecasting relies on meteorological data types like temperature, relative humidity, wind speed, rainfall etc.
- Each data point is associated with a timestamp, creating a sequence of observations that can be analysed over time.

Date	T_Max	T_Min	RH_1	RH_2	Wind_spe	sunshine	rainfall	evaporation
1/1/2016	16.8	12.4	100	98	13.75	0	8.5	1.16
1/2/2016	16.2	11.4	98	98	6.66	3	2.3	0.97
1/3/2016	18.8	12	92	98	5.92	5.8	0	1.58
1/4/2016	19.8	12.6	94	94	4.63	7.1	0	1.84
1/5/2016	20.2	14.8	96	74	5.52	10.4	0.2	1.47
1/6/2016	20	14.2	92	92	7.04	10.1	21.6	1.01
1/7/2016	16.6	13.6	98	98	5.56	10.7	2.5	0.11
1/8/2016	18.4	12.6	100	98	3.82	11	0	0.73
1/9/2016	19.8	12.6	85	98	2.65	11	0	0.4
1/10/2016	19	14.4	87	96	3.24	10	0.1	1.67
1/11/2016	19.2	11.4	94	98	3.25	10.6	0.4	0.26
1/12/2016	21.4	10	94	86	3.28	10.9	0	2.64

figure 1

(Seethaeliya weather data)

T_Min - Minimum temperature of the day (8.30a.m) in celcius

T_Max- Maximum temperature of the day(2.30p.m) in celcius

RH_1- Relative humidity (8.30a.m)

RH_2-Relative humidity (3.30p.m)

Wind_speed- Wind speed in Km/h

sunshine_hours- sunshine hours

rainfall-rainfall in mmh

evaporation-evaporation in means of evaporimeters



Stevenson cover



campbell-stokes sunshine recorder



Evaporometer

Why Time Series Analysis?

- Time series analysis can reveal recurring patterns.
- Techniques like ARIMA (Auto Regressive Integrated Moving Average) and RNN(Recurrent Neural Networks) can predict future weather conditions.

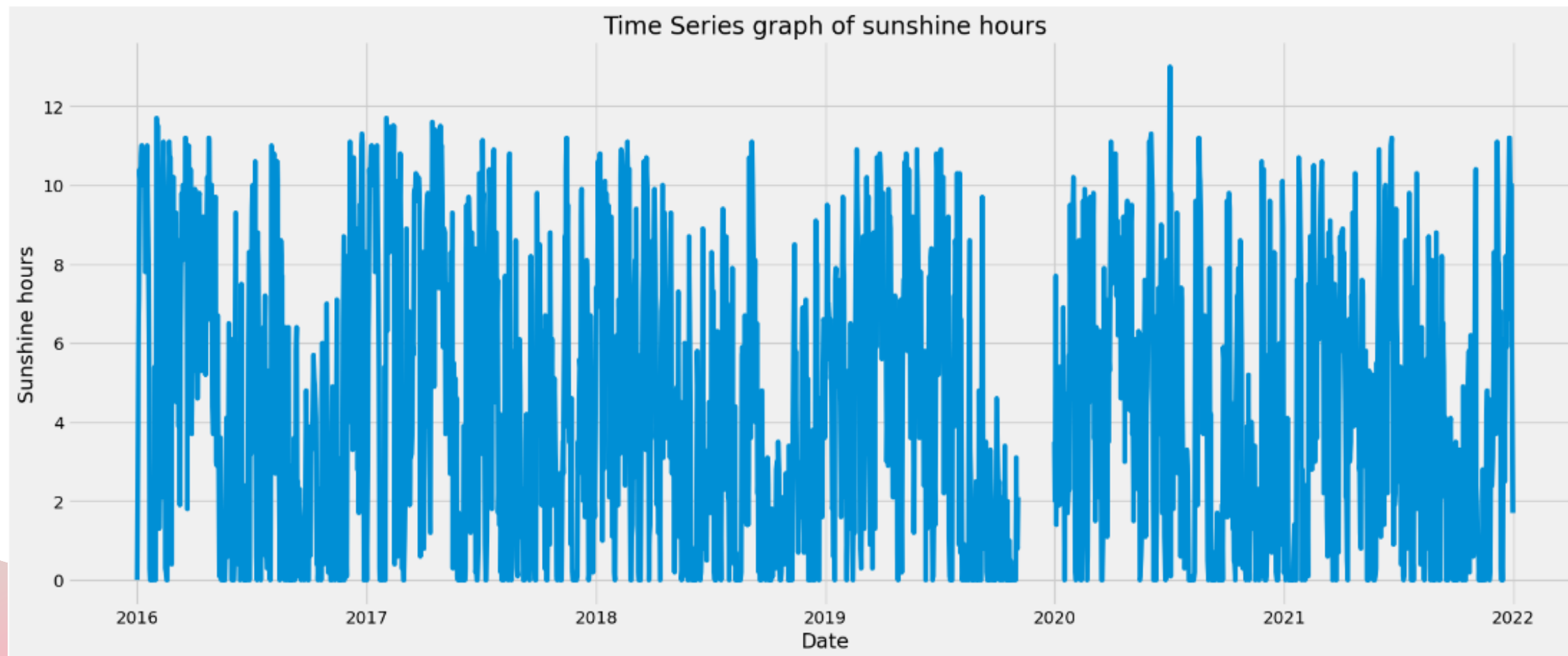


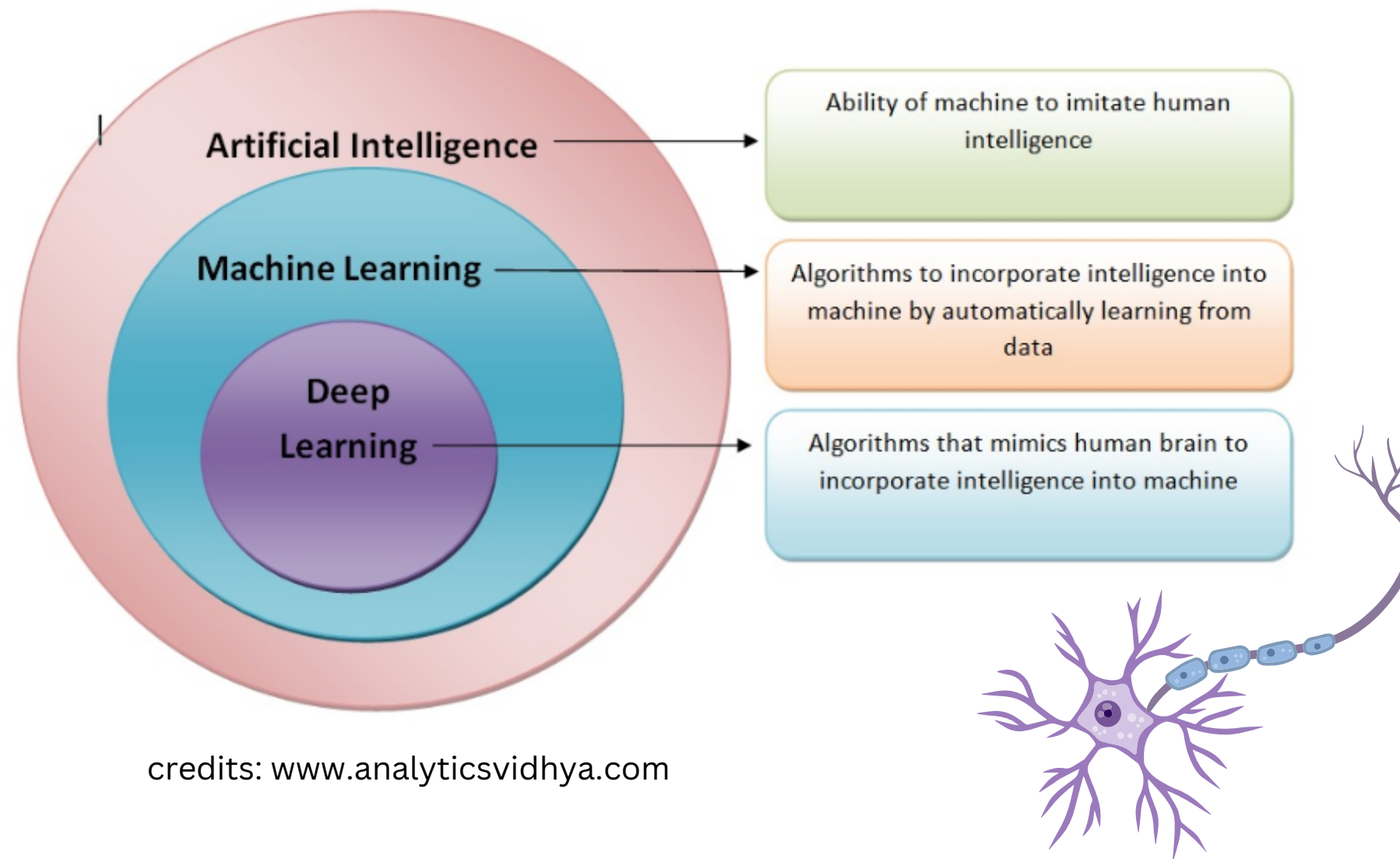
figure 2
(Time series graph of sunshine hours)



Numerical Weather Prediction Models (NWP Models)

- GFS (Global Forecasting Service)
- ECMWF(European Centre for Medium-range Weather Forecasting)

What is RNN?

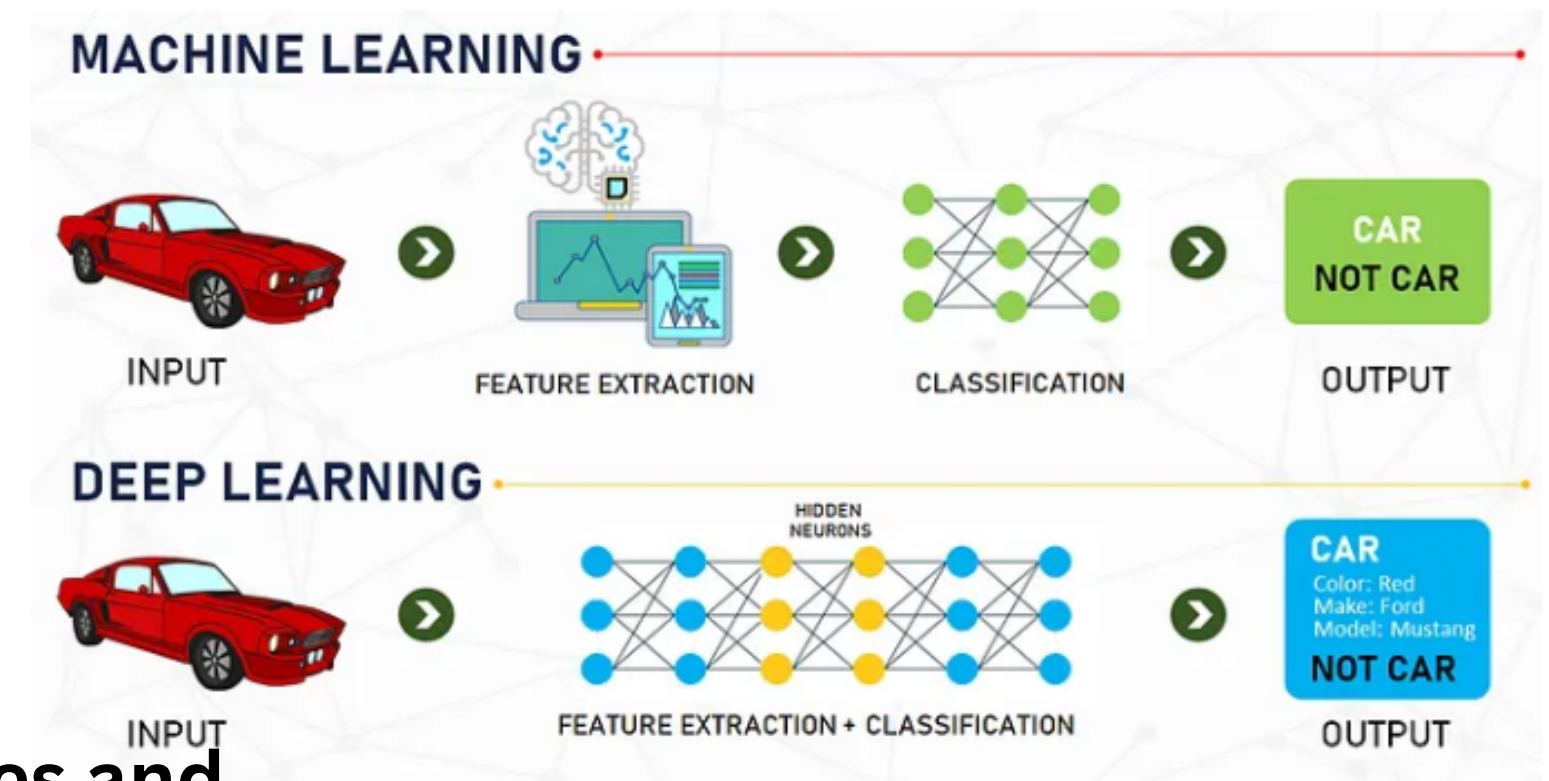


credits: www.analyticsvidhya.com

- RNN uses LSTM layers.
- RNN is able to capture temporal dependencies and sequences.

- In deep learning we use neural networks.

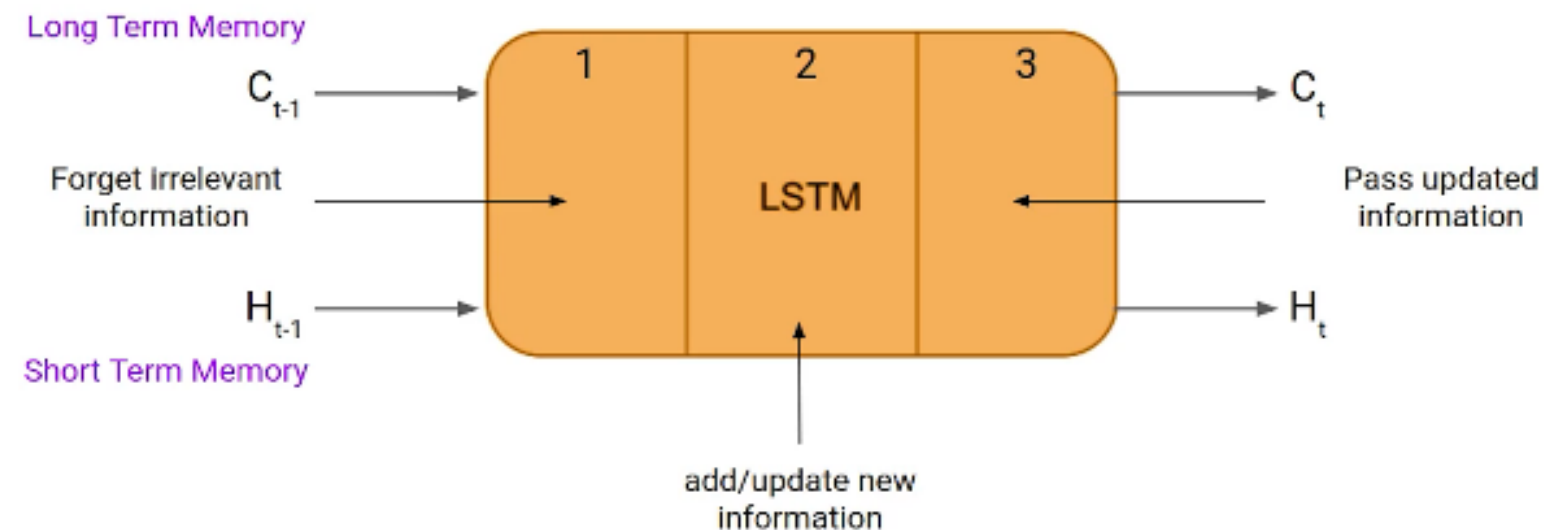
eg: ANN, RNN, CNN



credits: dltilabs

LSTM (Long Short Term Memory)

- Specialised type of RNN



- consists of Forget gate, Input gate and Output gate
- Can address the vanishing gradient problem in RNN (Remembers Long term dependencies)

- Applications of LSTM

eg: Generating Text
Speech Recognition
Sentiment Analysis



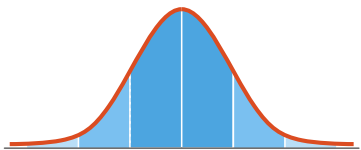
Building the RNN/LSTM Model

1.Data reading and Preprocessing

- Handling missing data
- Scaling the data (Normalization)
-

```
[83]: ##Scaling the data
scaler=MinMaxScaler(feature_range=(0,1))
sc=scaler.fit_transform(dataset5)
sc
```

```
[83]: array([[0.
          0.23076923],
          [0.44615385],
          ...,
          [0.76923077],
          [0.40769231],
          [0.13076923]])
```



```
[84]: #Create the training data set
##create the scaled training data set
train_data=sc[0:train_data_len,:]
##split the data into x_train and y_train data sets

x_train=[] ##features
y_train=[] ##target

for i in range(60,len(train_data)):
    x_train.append(train_data[i-60:i,0])
    y_train.append(train_data[i,0])
    if i<=60:
        print(x_train)
        print(y_train)
        print()

[0.5923076923076923, 0.23076923, 0.44615385, 0.54615385, 0.8
0.77692308, 0.82307692, 0.84615385, 0.84615385, 0.76923077,
0.81538462, 0.83846154, 0.82307692, 0.6
0.66153846,
0.60769231, 0.84615385, 0.73076923, 0.60769231, 0.40769231,
0.02307692, 0.
0.
0.
0.03846154, 0.12307692, 0.41538462, 0.
0.42307692, 0.9
0.48461538, 0.88461538, 0.72307692,
0.1
0.27692308, 0.26923077, 0.28461538, 0.30769231,
0.16153846, 0.78461538, 0.85384615, 0.75384615, 0.72307692,
0.02307692, 0.01538462, 0.
0.52307692,
0.63076923, 0.85384615, 0.82307692, 0.82307692, 0.53846154,
0.03076923, 0.74615385, 0.61538462, 0.78461538, 0.4
]]
```

2.Model architecture

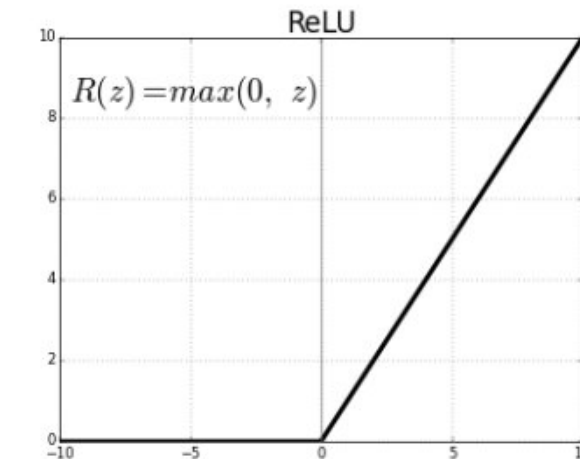
- Stack LSTM layers.
- Use activation functions

Importing the libraries

```
[69]: import numpy as np
import pandas as pd
import math
import matplotlib.pyplot as plt

from keras.models import Sequential
import matplotlib.patches as mpatches
from keras.layers import Dense
from keras.layers import Dropout
from keras.layers import LSTM
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error

import seaborn as sns
```



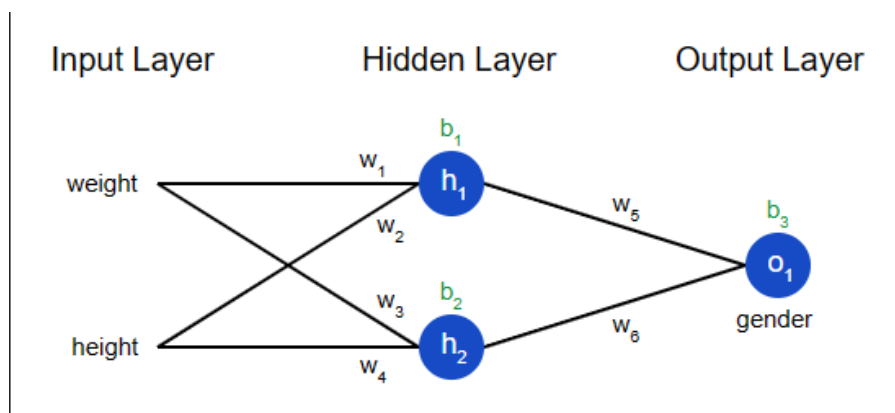
```
model=Sequential()
model.add(LSTM(50,activation='relu',return_sequences=True,input_shape=(x_train.shape[1],1)))
model.add(LSTM(50,activation='relu',return_sequences=False))
model.add(Dense(25,activation='relu'))
model.add(Dense(1))
```

Building the RNN/LSTM Model

3. Loss Function selection

- MSE for regression task.

```
#compile the model  
model.compile(optimizer='adam',loss='mean_squared_error')
```

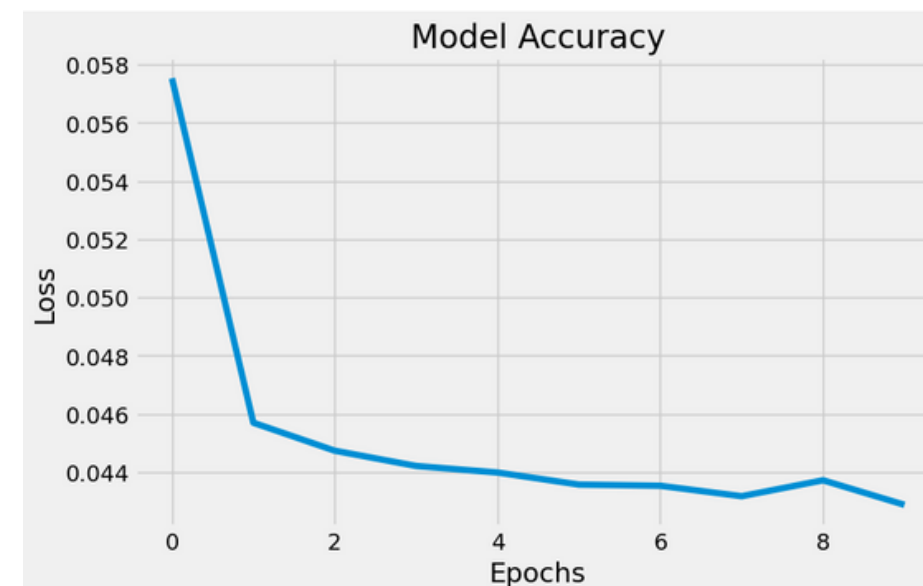


$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_{\text{true}} - y_{\text{pred}})^2$$

4. Training the model

- Backpropagation to minimize the loss function.

```
##train the model  
history = model.fit(x_train,y_train,epochs=10,batch_size=1)
```



Evaluating the Model

1. Mean absolute error
2. **Root Mean squared error**
3. skill scores

```
[93]: ##calculate RMSE
      ##Lower the RMSE higher the accuracy
      rmse=np.sqrt(np.mean(((predictions-y_test)**2)))
      rmse
```

```
[93]: 2.778708785625401
```



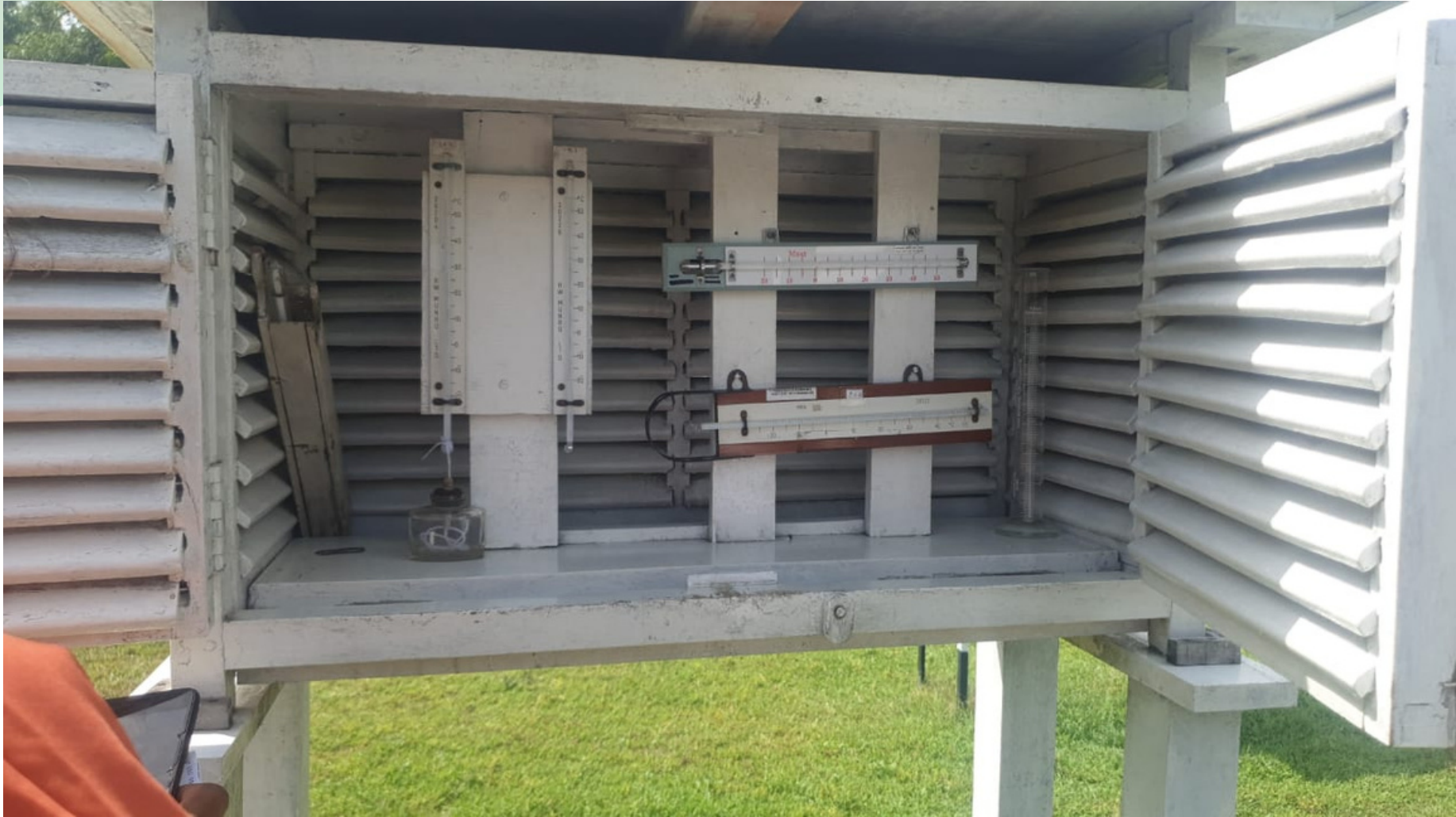
Future scopes

- RNN can be Integrated with satellite imagery and remote sensing data to make more accurate predictions
- Predict El-nino situations.
- Storm tracking and predictions using RNN



Special Thanks: Mr.K.D. Sujeewa (Deputy Director(Meteorologist))
Mr. Meril Mendis (Deputy Director at NMC)
Mr. Jagath(Meteorologist) and Mr. Nirmal
Mr. Sanjeewa

Thank You.





Month	Date	Dry	Wet	Dry	Wet	Max	Min	5 cm	10 cm	20 cm	30 cm	5 cm	10 cm	20 cm	30 cm	RH		Wind Speed	Sunshine Hours	Evaporation
		8.30 AM		15.30 PM				8.30 AM				15.30 PM				08.30 AM	15.30 PM			
January	1	15.0	14.0	14.6	14.4	NA	12.6	16.2	17.0	17.8	NA	18.4	18.2	18.0	NA	89	98	5.70	0.4	2.03
	2	15.0	14.6	15.4	15.2	NA	13.6	16.8	17.4	18.0	NA	17.6	18.0	18.0	NA	96	98	10.20	0.5	0.52
	3	15.6	15.4	15.2	15.0	NA	14.2	17.4	18.0	18.2	NA	18.0	18.2	18.2	NA	98	98	9.70	0.0	0.29
	4	14.6	14.4	16.2	16.0	NA	12.6	17.2	17.8	18.0	NA	18.4	18.2	18.0	NA	98	98	5.20	0.4	0.47
	5	16.6	15.6	18.0	17.4	NA	11.6	16.6	17.2	17.8	NA	19.0	17.6	18.0	NA	90	94	3.60	0.9	0.92
	6	16.8	15.8	16.4	15.6	NA	13.6	16.2	17.0	17.8	NA	18.4	19.2	18.0	NA	90	92	5.40	4.1	3.70
	7	17.2	14.6	16.8	15.2	NA	12.2	16.4	17.2	17.8	NA	19.6	19.4	18.2	NA	75	84	3.90	0.5	0.44
	8	15.8	14.8	17.6	16.4	NA	13.4	16.8	17.6	18.2	NA	19.6	19.2	18.4	NA	90	88	3.60	0.0	1.10
	9	15.4	14.6	16.4	16.2	NA	13.4	17.2	18.0	18.2	NA	18.4	18.2	18.2	NA	92	98	3.80	0.0	0.47
	10	14.8	14.6	15.0	14.6	NA	14.0	16.6	17.2	17.8	NA	17.2	17.8	17.8	NA	98	96	8.20	0.0	0.30
	11	15.6	15.2	16.4	16.2	NA	12.6	16.6	17.2	17.6	NA	18.4	18.2	17.8	NA	96	98	3.10	0.0	0.33
	12	15.8	15.6	16.4	16.0	NA	14.0	16.4	17.2	17.6	NA	18.2	18.2	17.8	NA	98	96	8.40	0.0	0.64
	13	15.2	15.2	16.8	16.2	NA	13.4	17.2	17.8	17.8	NA	18.8	18.6	17.8	NA	100	94	5.10	0.0	0.41
	14	15.8	15.4	16.8	16.4	NA	14.0	17.2	17.8	18.0	NA	18.4	18.4	18.0	NA	96	96	4.10	0.2	1.09
	15	15.4	14.4	17.2	16.6	NA	12.4	17.0	17.8	18.0	NA	19.2	19.0	18.2	NA	89	94	2.30	0.0	1.51
	16	17.6	15.4	18.0	16.2	NA	11.8	16.6	17.6	17.8	NA	18.6	18.4	18.0	NA	79	83	4.50	0.8	1.90
	17	15.6	14.6	16.6	15.0	NA	12.8	16.8	17.6	18.0	NA	19.0	18.8	18.0	NA	90	84	5.40	1.4	1.97
	18	16.6	15.6	16.4	16.2	NA	11.0	17.2	17.8	18.2	NA	19.2	19.0	18.2	NA	90	98	4.90	0.6	0.15
	19	17.4	15.0	16.8	15.8	NA	13.4	16.6	17.6	18.2	NA	19.6	19.4	18.2	NA	77	90	2.90	0.0	1.74
	20	18.0	16.0	19.6	17.6	NA	13.6	17.6	18.4	18.6	NA	21.6	21.2	19.2	NA	81	81	5.70	0.0	2.63
	21	17.6	15.8	19.2	16.8	NA	12.0	18.4	19.0	19.2	NA	22.0	21.6	19.4	NA	82	78	5.80	7.6	3.27

